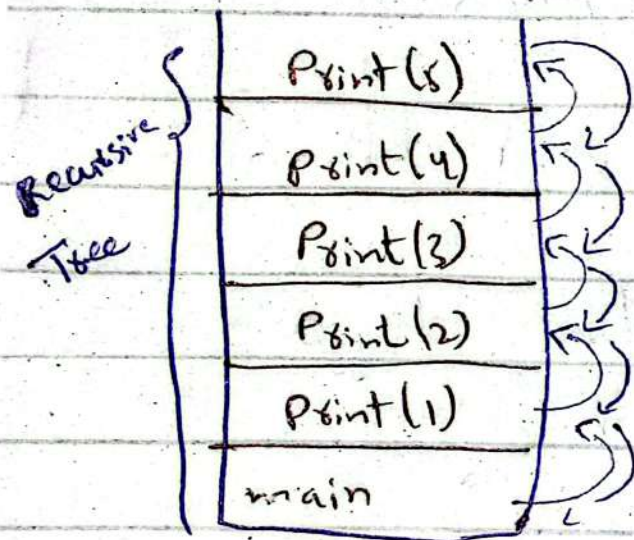


Introduction to Recursion:-

A function calling itself.

A function will wait till the next function to resolve. (ex Number Example Program).



Example is in Number Example Recursion

the main function waits in stack memory till $\text{Print}(1)$ function resolve & $\text{Print}(1)$ function wait till

$\text{Print}(2)$ resolve & so-on then when

$\text{Print}(5)$ function works done then it will go where it were called so it were called

In $\text{Print}(4)$ function same for $\text{Print}(4)$ till

main & apparently removes this way from stack.

Why do we need Recursion?

- * It helps us in solving bigger complex Problems in a simple way.
- * You can convert Recursion solutions into iteration & vice versa.
- * Space Complexity is not constant b/c of recursive calls.
- * It helps us in breaking down bigger Problems into smaller problems.

Q11 Find n^{th} Fibonacci number.

0th	1st	2nd	3rd	4th	5th	6th	7th	8th	
0	1	1	2	3	5	8	13	...	N

L> the addition of 2 previous numbers

just start from 0 & 1 then follow the

This is known as
→ recursive
relation

$$\text{Fibo}(N) = \text{fibo}(N-1) + \text{fibo}(N-2)$$

ex,

$\text{Fibo}(5)$ (Recursive Tree for
fibonacci number)

$\text{fibo}(4)$

+

$\text{fibo}(3)$

$\text{fibo}(3) + \text{fibo}(2)$

$\text{fibo}(2) + \text{fibo}(1)$

$\text{fibo}(2) + \text{fibo}(1)$

$\text{fibo}(1) + \text{fibo}(0)$

$\text{fibo}(1) + \text{fibo}(0)$

$\text{fibo}(1) + \text{fibo}(0)$

$\text{fibo}(1) + \text{fibo}(0)$

$\text{fibo}(1)$

$\text{fibo}(0)$

$\text{fibo}(0)$

$\text{fibo}(0)$

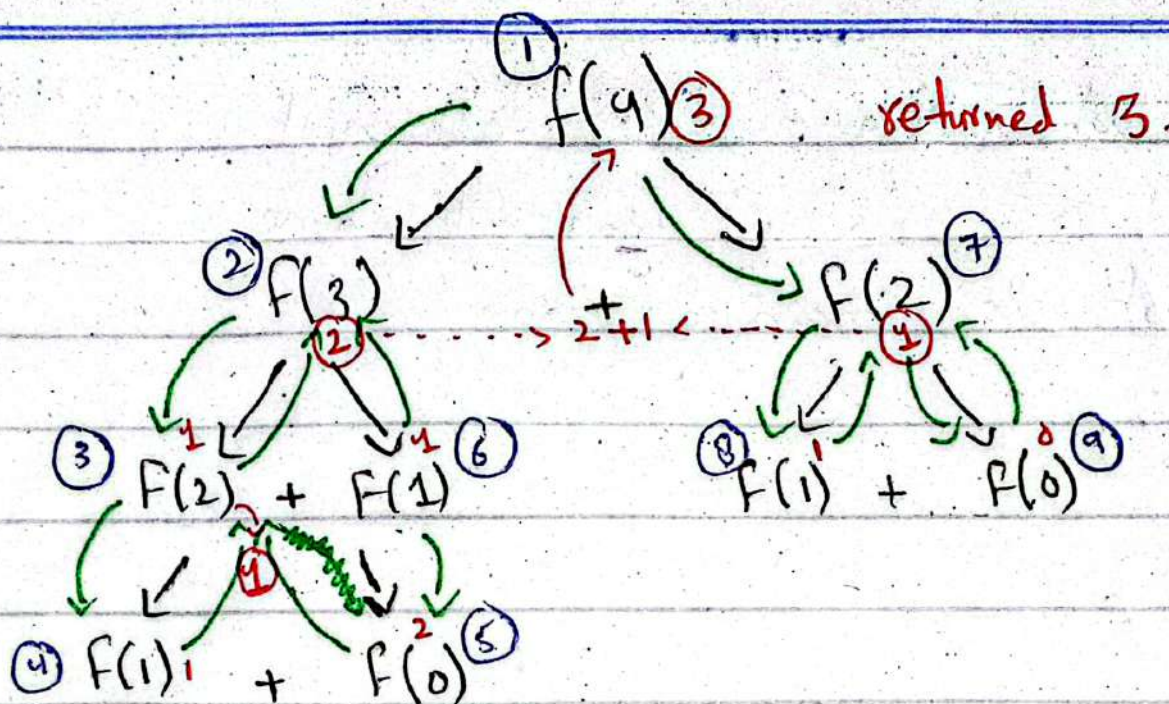
$\text{fibo}(1)$

How to Understand & Approach a Problem:-

- ① Identify if u can break down problem into smaller problems.
- ② Write the recurrence relation if needed.
- ③ Draw the Recursive Tree.
- ④ About the Tree:
 - * See the flow of functions, how they are getting in stack.
 - * Identify and focus left tree calls and right tree calls.
 - * Draw the tree and pointers again and again using pen & paper.
 - * Use a debugger to see the flow.
- ⑤ See how the values ^{& what types of values} are returned at each step.

See ~~the~~ where two function call will
come out of, and you will come
out of the main function.

Recursive Tree:



Variables:-

- 1) Arguments
- 2) Return type
- 3) Body of function

equation that defines the terms using previous terms.

Types of Recurrence Relation.

Linear recurrence relations.

e.g. Fibonacci number

Subtracting $(N-1) + (N-2)$
reducing it linearly.

→ Inefficient.

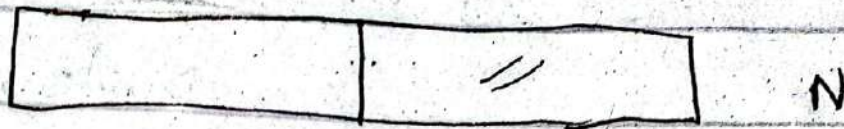
Divide and Conquer recurrence relation

e.g. binary search

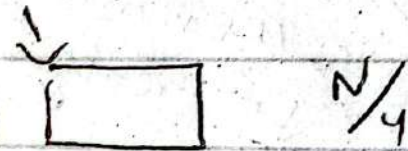
→ (search space is reduced by a factor),
e.g. $N/2$, $N/4$.

→ search spaces are reduced much faster in divide & conquer recurrences.
 $N/2$ faster than $N-1$.

Binary search with Recursion:-



1) Comparing the elements with target element $\rightarrow O(1)$



2) Dividing into 2-half.

⋮
till the target element (N) found.

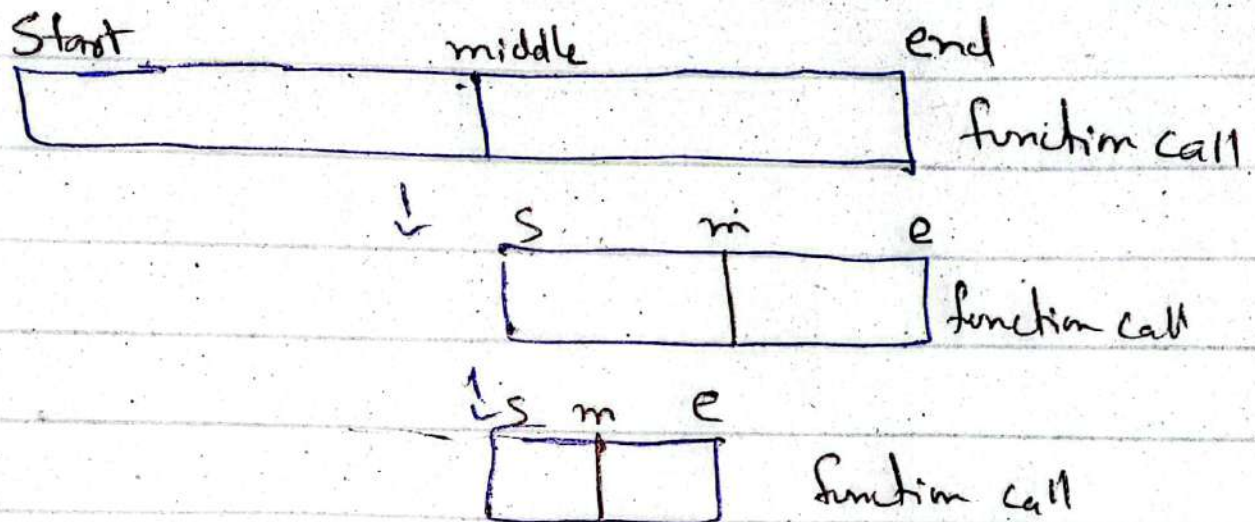
$$F(N) = O(N) + F(N/2) \quad \left\{ \begin{array}{l} \text{recurrence} \\ \text{relation} \end{array} \right.$$

$\downarrow$ Comparing in constant time $\downarrow$ Search in the array of size $N/2$.

Binary search

Continuation of where to take which variables?

in Argument in body in return type.



What are the 2 variables that will be used in each recursive call?

Passed $\text{Ans} \rightarrow$ Start and end, so we will use them in method Argument

$\rightarrow$ What about the middle variable?

$\rightarrow$ It's not beneficial for

future recursive calls so put

it in the body.